Ordenamiento por Intercambio (Bubble Sort), del juego de tarjetas al código

Objetivo

Comprender el algoritmo de **Bubble Sort** mediante la simulación con tarjetas. Después, diseñar una solución en **pseudocódigo** e implementarla en **Java**.

Materiales

- Tarjetas con números o nombres (mínimo 5). Ejemplo: 34, 12, 25, 8, 19.
- Hojas para registrar las pasadas.
- Computadora con JDK instalado y Bloc de Notas

Fase 1: Simulación con tarjetas (se conserva de la actividad original)

1. Formen equipos de 5 o más personas. Cada integrante es una tarjeta y se colocan en fila.
2. Comparen pares adyacentes de izquierda a derecha. Si la tarjeta de la izquierda es mayor que la de la derecha, intercambian lugares.
3. Recorrer toda la fila es **una pasada**. Repitan las pasadas hasta que ya no haya intercambios.
4. **Registren en una tabla** el estado de la fila al terminar cada pasada, el número de comparaciones y el número de intercambios.

Fase 2: Propuesta de solución en pseudocódigo (en binas)

A partir del registro de la Fase 1, escriban su propio pseudocódigo. Debe responder a estas preguntas:

- ¿Qué estructura de datos representa la fila de tarjetas?
  una lista de como casillas en la memoria de la computadora, donde cada tarjeta ocupa una posición fija como 1, 2, 3, etc, para que puedas encontrarla y compararla fácilmente
- ¿Qué ciclo representa las **pasadas** y cuál representa las **comparaciones** dentro de una pasada?
  el ciclo externo es cada vez que caminas desde el inicio hasta el final de la fila, el ciclo interno es cuando te vas parando frente a cada pareja de alumnos para ver quién es como el más alto
- ¿Cómo se realiza el **intercambio** de dos elementos sin perder uno de ellos?
  se usa una variable alterna o sea que funciones como una mesa , guardas el número de la izquierda en la mesa, mueves el número de la derecha a la izquierda, y luego agarras el número de la mesa y lo pones a la derecha.
- ¿Cómo sabe el algoritmo que **ya terminó**, es decir, cuándo detenerse antes de completar todas las pasadas?
  Pues si en una vuelta completa no se hace ningún intercambio, el algoritmo se da cuenta de que ya todo está ordenado

- ¿Cuántos elementos hay que comparar en cada pasada? Piensen en qué tarjetas ya quedaron en su lugar definitivo.
  en cada pasada se compara un elemento menos, los números más grandes van quedando fijos al final así que la fila que debes revisar se va haciendo cada vez más corta

Entregable: el pseudocódigo en hoja o archivo, con nombres de variables claros y sangría.

```
Algoritmo OrdenamientoTarjetas
    Dimension tarjeta[5]

 //Agarras 5 espacios en la memoria para poner tus tarjetasen la memoria
    // llenar las tarjetas
    Para i = 1 Hasta 5 Hacer
        Escribir "Dame tarjeta ", i
        Leer tarjeta[i]
    FinPara

    // pasadas y comparaciones

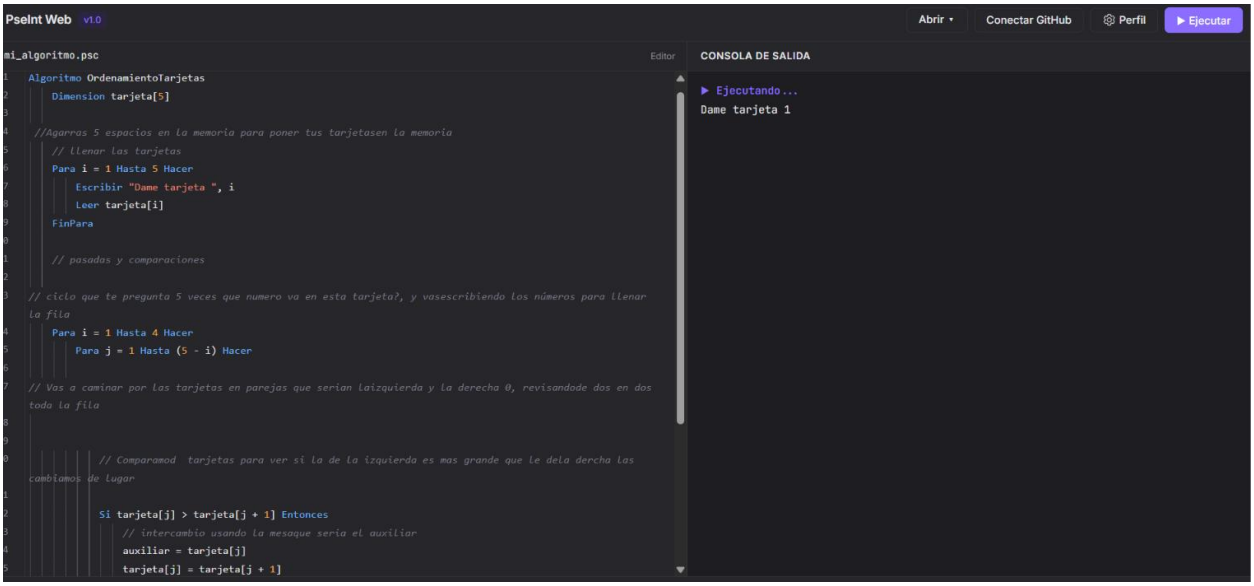
// ciclo que te pregunta 5 veces que numero va en esta tarjeta?, y vasescribiendo los números
para llenar la fila
    Para i = 1 Hasta 4 Hacer
        Para j = 1 Hasta (5 - i) Hacer

// Vas a caminar por las tarjetas en parejas que serian laizquierda y la derecha 0, revisandode
dos en dos toda la fila


            // Comparamod  tarjetas para ver si la de la izquierda es mas grande que le dela dercha
las cambiamos de lugar

            Si tarjeta[j] > tarjeta[j + 1] Entonces
                // intercambio usando la mesaque seria el auxiliar
                auxiliar = tarjeta[j]
                tarjeta[j] = tarjeta[j + 1]
                tarjeta[j + 1] = auxiliar
            FinSi
        FinPara
    FinPara
```

```
    // 3. Mostrar resultado d ecomo quedo
    Para i = 1 Hasta 5 Hacer
        Escribir tarjeta[i]
    FinPara
FinAlgoritmo
```



Fase 3: Implementación en Java

1. Traduzcan su pseudocódigo a un programa en Java con un método bubbleSort(int[] arreglo).
2. El programa debe mostrar el arreglo **después de cada pasada**, para compararlo con el registro de la Fase 1.
3. Debe contar e imprimir el **total de comparaciones, intercambios y pasadas**.
4. Prueben con el arreglo {34, 12, 25, 8, 19}, con uno ya ordenado y con uno en orden inverso.
5. Variación opcional: ordenar un arreglo de String alfabéticamente con compareTo.

```
import java.util.Arrays;

public class OrdenamientoBurbuja {

    public static void bubbleSort(int[] tarjeta) {
        // aquí guuardamos cuántas tarjetas tenemos en total en la fila
        int n = tarjeta.length;

        // contadores para llevar el registro que pide la tarea
        int pasadas = 0;
        int comparaciones = 0;
        int intercambios = 0;

        System.out.println("Arreglo inicial: " + Arrays.toString(tarjeta));

        // ciclo externo, cuenta cada pasada completa que damos por la fila
```

```
        for (int i = 0; i < n - 1; i++) {
            pasadas++;

            // ciclo interno , caminamos por la fila comparando de par en par, izquierda y derecha
            for (int j = 0; j < n - 1 - i; j++) {
                comparaciones++; // contamos cada comparación que hacemos

                // si la tarjeta de la izquierda es más grande que la de la derecha, están al revés
                if (tarjeta[j] > tarjeta[j + 1]) {

                    // Usamos la mesa que es el auxiliar para hacer el cambio sin perder datos
                    int auxiliar = tarjeta[j];        //guardamos temporalmente el número de la izquierda en la mesa
                    tarjeta[j] = tarjeta[j + 1];  // psamos el número de la derecha al lugar de la izquierda
                    tarjeta[j + 1] = auxiliar;    // ponemos el número que estaba en la mesa al lugar de la derecha

                    intercambios++; // Contamos este intercambio
                }
            }

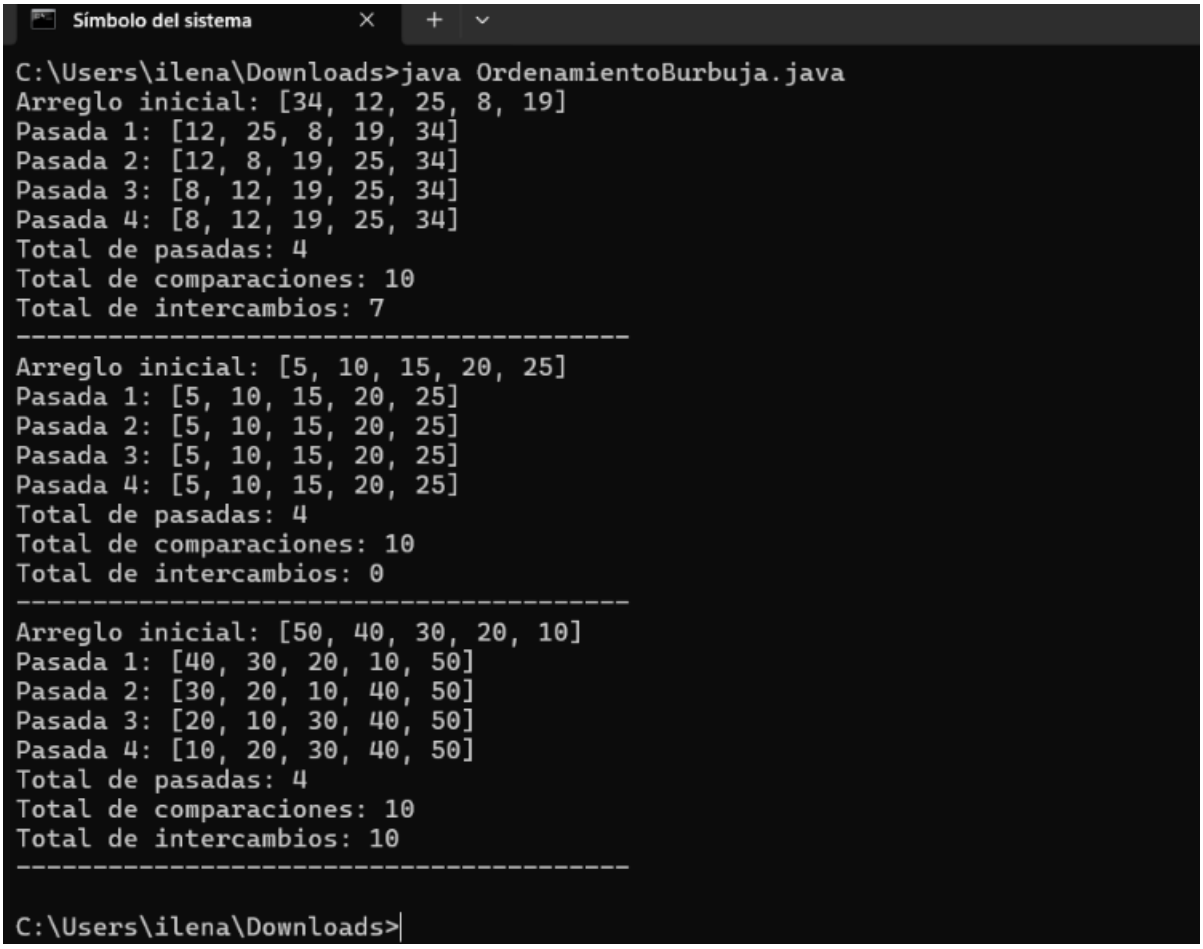
            // mostramos en pantalla cómo quedó la fila de tarjetas al terminar esta pasada
            System.out.println("Pasada " + pasadas + ": " + Arrays.toString(tarjeta));
        }

        // imprimimos los totales finales que pide la rúbrica de la tarea
        System.out.println("Total de pasadas: " + pasadas);
        System.out.println("Total de comparaciones: " + comparaciones);
        System.out.println("Total de intercambios: " + intercambios);
        System.out.println("----------------------------------------");
    }

    public static void main(String[] args) {
        // Prueba 1: El arreglo revuelto que pide exactamente la tarea de la foto
        int[] t1 = {34, 12, 25, 8, 19};
        bubbleSort(t1);

        // Prueba 2: Un arreglo que ya viene ordenado de inicio
        int[] t2 = {5, 10, 15, 20, 25};
        bubbleSort(t2);

        // Prueba 3: Un arreglo que está totalmente al revés (orden inverso)
        int[] t3 = {50, 40, 30, 20, 10};
        bubbleSort(t3);
    }
}
```


Símbolo del sistema
C:\Users\ilena\Downloads>java OrdenamientoBurbuja.java
Arreglo inicial: [34, 12, 25, 8, 19]
Pasada 1: [12, 25, 8, 19, 34]
Pasada 2: [12, 8, 19, 25, 34]
Pasada 3: [8, 12, 19, 25, 34]
Pasada 4: [8, 12, 19, 25, 34]
Total de pasadas: 4
Total de comparaciones: 10
Total de intercambios: 7

Arreglo inicial: [5, 10, 15, 20, 25]
Pasada 1: [5, 10, 15, 20, 25]
Pasada 2: [5, 10, 15, 20, 25]
Pasada 3: [5, 10, 15, 20, 25]
Pasada 4: [5, 10, 15, 20, 25]
Total de pasadas: 4
Total de comparaciones: 10
Total de intercambios: 0

Arreglo inicial: [50, 40, 30, 20, 10]
Pasada 1: [40, 30, 20, 10, 50]
Pasada 2: [30, 20, 10, 40, 50]
Pasada 3: [20, 10, 30, 40, 50]
Pasada 4: [10, 20, 30, 40, 50]
Total de pasadas: 4
Total de comparaciones: 10
Total de intercambios: 10

C:\Users\ilena\Downloads>

Fase 4: Reflexión

- ¿Coincide la salida del programa con lo que registraron con las tarjetas?
  si, porque hace exactamente lo mismo, va comparando los números de dos en dos y usando la mesa que es el auxiliar para acomodarlos de menor a mayor.
- ¿Qué cambios tuvo que hacer su pseudocódigo al pasarlo a Java?
  Solo cambiamos las palabras del pseudocódigo (Para, FinPara) por la estructura de Java con llaves {} y los tipos de datos (int[]), pero la lógica es igual en si

- ¿Qué pasa cuando el arreglo ya está ordenado? ¿Sigue comparando?
  No , el programa se da cuenta de que no hubo ningún cambio en la primera vuelta y se detiene antes de tiempo para ahorrar trabajo
- ¿Es eficiente con listas muy grandes? Relacionen su respuesta con el número de comparaciones (O(n²)).
  No es eficiente para listas gigantes. Como su complejidad es de $O(n^2)$, si la lista crece, el número de comparaciones e intercambios se multiplica muchísimo y se vuelve lento.

Lista de cotejo (5 puntos)

#	Criterio	Cumple	No cumple	Observaciones
1	**Simulación y registro:** la tabla de pasadas con tarjetas es completa y correcta (estado de la fila, comparaciones e intercambios por pasada).			
2	**Pseudocódigo:** es claro, está estructurado y contiene los dos ciclos (pasadas y comparaciones), la condición de comparación, el intercambio con variable auxiliar y la condición de paro.			
3	**Implementación en Java:** el programa compila, ejecuta y ordena correctamente el arreglo de ejemplo y los casos de prueba (ordenado e inverso).			
4	**Coherencia pseudocódigo–código:** el programa respeta la lógica del pseudocódigo propuesto y muestra el arreglo tras cada pasada, junto con los contadores.			
5	**Reflexión y buenas prácticas:** responde las preguntas de reflexión con argumentos, incluida la eficiencia, y el código usa nombres significativos, sangría y comentarios.			